

CERTIFICATION

Exam objectives map

The May 2026 exam guide mapped to course modules and labs

Contents

OVERVIEW	About the exam	3
SECTION 1	Databricks Intelligence Platform (6%)	4
SECTION 2	Data Ingestion and Loading (21%)	5
SECTION 3	Data Transformation and Modeling (22%)	8
SECTION 4	Working with Lakeflow Jobs (16%)	11
SECTION 5	Implementing CI/CD (10%)	12
SECTION 6	Troubleshooting, Monitoring, and Optimization (10%)	14
SECTION 7	Governance and Security (15%)	16
PRACTICE	Sample question patterns	19
PLAN	Two-week study plan	20

This map links every objective in the official Databricks Certified Data Engineer Associate exam guide to the modules and labs of this course, and lists what you need to know for each one. To revise, take one section at a time: reopen the notebooks listed under *Where in the course*, then check that you can explain every bullet point below it without notes: the bullets are what you need to know for the exam. Where an objective goes beyond the course notebooks, the explanation here is complete, so you can revise it from this document alone.

OVERVIEW

About the exam

Item	Detail
Exam guide version	May 4, 2026
Questions	45 scored multiple-choice questions; unscored items may be added, with extra time included
Time limit and fee	90 minutes; USD 200 plus applicable taxes
Delivery	Online proctored or at a test centre; no test aids
Prerequisites	None required; course attendance and about six months of hands-on experience recommended
Validity	Two years; recertify by passing the current exam
Passing score	Not published; aim for at least 80% on practice questions

Exam guides change every few months. About two weeks before your exam, download the current guide from databricks.com/learn/certification and compare it with this map.

Section	Domain	Weight	Approx. questions	Course days
1	Databricks Intelligence Platform	6%	3	Day 1
2	Data Ingestion and Loading	21%	9	Days 1-2
3	Data Transformation and Modeling	22%	10	Days 1-2 + self-study
4	Working with Lakeflow Jobs	16%	7	Day 3
5	Implementing CI/CD	10%	4-5	Day 3
6	Troubleshooting, Monitoring, and Optimization	10%	4-5	Days 2-3
7	Governance and Security	15%	7	Day 3

Question counts are estimates: the weight multiplied by 45. The guide lists objectives without numbers; this map numbers them in guide order (2.3 is the third objective of Section 2) and shortens the wording. Course references name demo notebooks (`05_incremental_ingestion`), labs (`lab_05_streaming`, with guides and solutions in `notebooks/guides` and `notebooks/solution`) and supporting code in `materials/`.

SECTION 1

Databricks Intelligence Platform (6%)

1.1 Core components: architecture, Delta Lake, Unity Catalog

Where in the course: 00_intro, 01_platform_and_workspace, 03_delta_lake, lab_01_first_steps, lab_03_delta_dml

- Databricks manages the control plane (web application, job scheduler, Unity Catalog). Compute runs as serverless compute in the Databricks account or as classic compute in your cloud subscription. Data stays in cloud object storage.
- Delta Lake is Parquet files plus a `_delta_log` transaction log. It provides ACID transactions, schema enforcement, `DESCRIBE HISTORY` for auditing, and time travel with `VERSION AS OF`, `TIMESTAMP AS OF` and `RESTORE`.
- Unity Catalog has one metastore per region and a `catalog.schema.object` namespace. It governs tables, views, volumes, functions, models, connections and external locations, and records lineage and audit events. Volumes (`/Volumes/...`) replace the DBFS root and mounts for files.
- Scenario: rollback after a bad load, an audit trail and one source of truth for BI and AI all point to Delta Lake time travel governed by Unity Catalog.

1.2 Compute services: characteristics, limitations, cost models

Where in the course: 01_platform_and_workspace (compute resources, decision tree), 08_job_orchestration (serverless compute for jobs), lab_08_orchestration (closing check)

- Many analysts running ad hoc SQL with fast startup and controlled cost: a serverless SQL warehouse with scaling and auto-stop. Older exam items call this a high-concurrency cluster with autoscaling.
- Photon is on by default on SQL warehouses and serverless compute. Classic access modes are standard (formerly shared, multi-user) and dedicated (formerly single user). Instance pools shorten classic startup.

Compute	Best for	Cost model	Limitations
Serverless notebooks, jobs, pipelines	Most ETL and interactive work; starts in seconds	DBUs while running, no separate VM charge	Few settable Spark properties; no init scripts or compute libraries; query profile instead of Spark UI
SQL warehouse (serverless)	SQL, dashboards, BI, many concurrent users	DBUs while running; auto-stop	SQL only; adds clusters to handle concurrency
Job compute (classic)	Scheduled jobs needing a specific runtime, libraries or init scripts	Lower DBU rate plus VMs; ends with the run	Minutes to start
All-purpose compute (classic)	Interactive development	Higher DBU rate plus VMs	Idle clusters cost money; set auto-termination

SECTION 2

Data Ingestion and Loading (21%)

2.1 Ingestion patterns and sources

Where in the course: 02_data_ingestion (ingestion methods, data processing patterns, `read_files()`, CTAS), 02a_lakeflow_connect_demo, 05_incremental_ingestion, lab_02_ingestion, lab_05_streaming

- **Batch** reads everything on every run (`spark.read`, CTAS over `read_files()`). **Incremental** loads only new data, with state kept by COPY INTO load history, an Auto Loader checkpoint or a streaming table. **Streaming** uses Structured Streaming; `trigger(availableNow=True)` processes the backlog and stops.
- **Local files** go through the *Add data* upload (into a managed Delta table) or into a volume.
- **Standard connectors** (cloud storage with Auto Loader, `read_files` or COPY INTO; Kafka; SFTP) need pipeline code that you maintain.
- **Managed connectors** cover SaaS applications (Salesforce, Workday, ServiceNow, Dynamics 365), databases (SQL Server, MySQL, PostgreSQL, Oracle) and file services (SharePoint, Google Drive). You configure them in the UI, API or bundles, and they land incremental Unity Catalog streaming tables.

2.2 COPY INTO

Where in the course: 05_incremental_ingestion (COPY INTO, idempotency), lab_05_streaming Task 1

- Idempotent: files already loaded are skipped, and `COPY_OPTIONS ('force' = 'true')` reloads them. Select files with `PATTERN` or `FILES`.
- The source is a volume or a cloud URI inside an external location. It requires `MODIFY` on the table and `READ FILES` or `READ VOLUME` on the source.
- Suits SQL teams and scheduled loads of up to thousands of files. For millions of files or continuous arrival, use Auto Loader or a streaming table.

```
CREATE TABLE IF NOT EXISTS main.bronze.orders_raw;      -- schema comes from the files
COPY INTO main.bronze.orders_raw
FROM '/Volumes/main/landing/raw/orders/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

2.3 Auto Loader with schema enforcement and evolution

Where in the course: 05_incremental_ingestion (Auto Loader, schema evolution and rescued data), lab_05_streaming Tasks 2–6, BONUS_timestamps_autoloader_archive

- Enforce a schema with `.schema(...)` or `cloudFiles.schemaHints`. Without `cloudFiles.inferColumnTypes`, JSON and CSV columns are strings.
- “Batch mode” means `availableNow=True` from a scheduled job; the checkpoint gives exactly-once, incremental runs. Each stream needs its own checkpoint.

- File discovery: **directory listing** (default, no setup, lists the path each run) or **file notification** (event-based, scales to millions of files). The recommended form is file events on the external location with `cloudFiles.useManagedFileEvents = true`; the classic form is `cloudFiles.useNotifications = true`.
- In SQL: `CREATE OR REFRESH STREAMING TABLE t AS SELECT * FROM STREAM read_files('/Volumes/...', format => 'json')`.

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/Volumes/main/bronze/chk/orders_schema")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .load("/Volumes/main/landing/raw/orders/")
  .writeStream.option("checkpointLocation", "/Volumes/main/bronze/chk/orders")
  .trigger(availableNow=True).toTable("main.bronze.orders"))
```

Evolution mode	When new columns appear
addNewColumns (default without a schema)	Stream stops, stored schema is updated, the next run (for example a job retry) continues
rescue	Schema never changes; unexpected data goes to <code>_rescued_data</code>
failOnNewColumns	Stream fails until the schema is updated manually
none (default with a schema)	New columns are ignored

2.4 Configure Lakeflow Connect

Where in the course: 02_data_ingestion (Lakeflow Connect), 02a_lakeflow_connect_demo, guide *Lakeflow Connect* (lakeflow_connect_sqlserver)

- A **Unity Catalog connection** stores credentials (`CREATE CONNECTION` on the metastore to create it; `USE CONNECTION` to use it). The **ingestion pipeline** runs on serverless compute and writes streaming tables (`USE CATALOG, USE SCHEMA, CREATE TABLE`).
- Database connectors add an **ingestion gateway**. It runs continuously on classic compute and stages snapshots and changes in a volume, so it needs `CREATE VOLUME`.
- Setup path: **Data Ingestion**, then the connector, connection, source tables, destination and schedule. The API, CLI and bundles work too.
- Change capture: SQL Server uses change tracking (tables with a primary key) or CDC (tables without one) after an initial snapshot; SaaS connectors use API cursors. Options include SCD type 1 or 2, column selection, destination names and full refresh.

2.5 JDBC and REST clients in notebooks, orchestrated by jobs

Where in the course: 02_data_ingestion (JDBC/ODBC), 08_job_orchestration (widgets and parameters), lab_08_orchestration. The REST pattern is covered only here.

- Keep credentials in secret scopes; `dbutils.secrets.get` values are redacted in output.
- Parallel JDBC reads need all four of `partitionColumn`, `lowerBound`, `upperBound` and `numPartitions`; otherwise a single connection reads the whole table.
- JDBC is not incremental on its own: store a watermark (last `modified_at`) and use `MERGE` for upserts. Land raw REST responses in a volume and load them with Auto Loader.

- Orchestrate with a notebook task, job parameters, a schedule and retries for transient network errors. Prefer a managed connector or Lakehouse Federation when one exists.

```
orders = (spark.read.format("jdbc")
    .option("url", "jdbc:sqlserver://srv.database.windows.net:1433;database=sales")
    .option("dbtable", "dbo.orders")
    .option("user", dbutils.secrets.get("jdbc", "user"))
    .option("password", dbutils.secrets.get("jdbc", "password"))
    .option("partitionColumn", "order_id").option("lowerBound", 1)
    .option("upperBound", 5000000).option("numPartitions", 8)
    .load())
orders.write.mode("append").saveAsTable("main.bronze.orders_jdbc")

import requests
run_date = dbutils.widgets.get("run_date") # job parameter
resp = requests.get("https://api.example.com/v1/rates", params={"date": run_date},
    headers={"Authorization": f"Bearer {dbutils.secrets.get('api', 'token')}"},
    timeout=30)
resp.raise_for_status()
dbutils.fs.put(f"/Volumes/main/landing/raw/fx/{run_date}.json", resp.text, overwrite=True)
```

2.6 Choosing an ingestion method

Where in the course: 02a_lakeflow_connect_demo (section A), 02_data_ingestion (comparison of ingestion methods), 05_incremental_ingestion

Criteria: volume, frequency and latency, data types, Unity Catalog governance and maintenance effort. Start with the most managed option and move to custom code only when it cannot support the source.

Requirement	Choose
Files keep arriving in storage; schema drift	Auto Loader or a streaming table with <code>read_files</code>
Up to thousands of files, SQL team, idempotent reloads	COPY INTO
SaaS application or database with a managed connector	Lakeflow Connect managed connector
Kafka or another message bus	Structured Streaming or a pipeline with the Kafka connector
No Databricks connector, existing partner licence	Partner tool (Fivetran, Qlik, Informatica)
Custom API or unsupported database, small volume	Notebook with REST or JDBC in a job
Query without copying	Lakehouse Federation

2.7 Semi-structured and unstructured data via managed connectors

Where in the course: 02_data_ingestion (JSON reader options), 05a_transformations (complex types), lab_06_transformations. Managed file connectors are covered only here.

- The SharePoint and Google Drive managed connectors ingest **BINARYFILE** (one row per file with a `content` column and metadata, for PDFs, images and Office files; parse later with `ai_parse_document`) or structured formats (CSV, JSON, XML, EXCEL, PARQUET, AVRO, ORC), parsed into rows with Auto Loader schema evolution modes.

- Database and SaaS connectors map source types to Delta types (SQL Server XML becomes STRING); check with `DESCRIBE TABLE` before flattening.
- Nested JSON can land as one `VARIANT` column, which tolerates schema changes: set the `singleVariantColumn` option in Auto Loader, `read_files` or `COPY INTO`. Extract fields with `payload:customer.address.city::STRING`.
- Flatten structs with dot notation. `explode` turns array elements into rows and drops empty arrays; `explode_outer` keeps them. Parse JSON strings with `from_json(col, schema)` or `parse_json`. Keep raw nesting in bronze; flatten in silver.

SECTION 3

Data Transformation and Modeling (22%)

Objectives 3.1–3.5 are self-study in this course: work through `05a_transformations` and `lab_06_transformations` (a guide and a solution notebook are included), then check yourself against the points below and the Day 2 quiz. Objectives 3.6 and 3.7 are taught in class in modules 06 and 07.

3.1 Cleaning bronze into silver

Where in the course: `06_medallion_architecture` (silver layer), `materials/medallion/silver_*`, `02_data_ingestion` (DataFrame transformations), `lab_06_transformations`

- Read with `spark.read.table("main.bronze.orders")`. Handle nulls with `dropna(subset=[...])`, `fillna({...})`, `coalesce()` or `when().otherwise()`.
- Standardise types with `cast`, `to_date` and `to_timestamp` using explicit formats. ANSI mode (default on serverless and SQL warehouses) raises errors on invalid casts; `try_cast` returns NULL instead.
- Normalise strings with `trim` and `lower` before joining or deduplicating on them.
- Write with `saveAsTable`, `CREATE OR REPLACE TABLE ... AS SELECT` or `MERGE INTO` for upserts.

3.2 Joins and unions

Where in the course: `05a_transformations` (joins), `lab_06_transformations` Tasks 1–3, `lab_troubleshooting` Task 3, guide `pyspark_vs_sparksql`

- Join types: `inner`, `left`, `right`, `full`, `left_semi` (matching rows, left columns), `left_anti` (non-matching rows), `cross`.
- A list of key names returns one column per key; an expression such as `(a.k1 == b.k1) & (a.k2 == b.k2)` keeps both copies, so drop or alias them.
- Broadcast joins show `BroadcastHashJoin` in `explain()`; the SQL hint is `/*+ BROADCAST(p) */`. A wrong or truncated join key multiplies rows much like a cross join.
- PySpark `union()` and `unionAll()` are the same: they match by position and keep duplicates, so add `.distinct()` to remove them. SQL `UNION` removes duplicates and `UNION ALL` keeps them.

```

from pyspark.sql.functions import broadcast
multi = sales.join(targets, on=["store_id", "sale_date"], how="inner") # multiple keys
left = orders.join(customers, "customer_id", "left")
fast = orders.join(broadcast(products), "product_id") # no shuffle of orders
grid = stores.crossJoin(calendar) # n x m rows
both = orders_2025.union(orders_2026) # by position, keeps duplicates
names = orders_2025.unionByName(orders_2026, allowMissingColumns=True)

```

3.3 Columns, rows and table structure

Where in the course: 02_data_ingestion (select, withColumn, cast, filter), 05a_transformations, lab_06_transformations Task 4, 03_delta_lake (schema evolution)

- Columns: withColumn, withColumns({...}), withColumnRenamed, drop, select, selectExpr. Split strings with F.split("name", " ").getItem(0).
- Rows: filter or where with column expressions or SQL strings.
- Arrays: explode, posexplode, collect_list, collect_set; higher-order functions transform, filter and exists avoid exploding.
- Tables: ALTER TABLE ... ADD COLUMNS. RENAME COLUMN and DROP COLUMN require column mapping ('delta.columnMapping.mode' = 'name'). Write with mergeSchema to add columns.

3.4 Deduplication and aggregation

Where in the course: 05a_transformations (deduplication, approximate and descriptive aggregations), lab_06_transformations Tasks 3 and 5

- distinct() compares whole rows; dropDuplicates(["key"]) keeps an arbitrary row per key. Keep the latest row with row_number(), or QUALIFY in SQL.
- count("*") counts all rows and count(col) skips nulls. approx_count_distinct (HyperLogLog++, default 5% relative error) is far cheaper than an exact distinct count.
- describe() returns count, mean, stddev, min and max; summary() adds the 25%, 50% and 75% percentiles. In streaming, use dropDuplicatesWithinWatermark.

```

w = Window.partitionBy("order_id").orderBy(F.col("_ingest_ts").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
df.agg(F.countDistinct("customer_id"), F.approx_count_distinct("customer_id")).show()

```

3.5 Basic tuning parameters

Where in the course: 05a_transformations (tuning configurations), lab_06_transformations Task 7, lab_troubleshooting Task 4

- Serverless accepts only a few properties, among them spark.sql.shuffle.partitions, spark.sql.session.timeZone, spark.sql.ansi.enabled and spark.sql.files.maxPartitionBytes. Setting an unsupported property fails the job, so use broadcast() hints instead.
- Adaptive query execution (on by default) coalesces partitions, switches to broadcast joins and splits skewed partitions at run time.

- Re-measure with one change at a time on the same data and compute. Time an action such as `df.write.format("noop").mode("overwrite").save()`, compare shuffle sizes, task counts and spill, then restore the setting.

Parameter	Controls	Default and notes
<code>spark.sql.shuffle.partitions</code>	Partitions after joins and aggregations	200 on classic, auto on serverless; settable at runtime; lower for small data
<code>spark.default.parallelism</code>	Default partitions for RDD operations	Total cores; DataFrame shuffles ignore it
<code>spark.executor.memory</code>	Heap per executor	Set before startup, usually by choosing the node type; too low causes spill and OOM
<code>spark.driver.memory</code>	Driver heap	Fixed at startup; matters for <code>collect()</code> , <code>toPandas()</code> and broadcasts
<code>spark.sql.autoBroadcastJoinThreshold</code>	Largest table broadcast automatically	10 MB; -1 disables; raise for bigger dimensions

3.6 Gold objects: tables, views, materialized views, streaming tables

Where in the course: `07_lakeflow_pipelines` (table types, `STREAM()` vs direct reference), `06_medallion_architecture` (gold), `lab_07_lakeflow_pipeline` Task 4, `10_governance_and_security` (dynamic views)

- Streaming tables need append-only sources, and a materialized view that reads `STREAM(...)` is invalid. In Databricks SQL, a serverless pipeline maintains both object types.
- All four are Unity Catalog securables; give BI users `SELECT`.

Object	Stores data	Stays current by	Use
Table	Yes	Your writes: <code>INSERT</code> , <code>MERGE</code> , <code>overwrite</code>	Dimensions, SCD, full control
View	No	Running the query at read time	Light logic, security layer
Materialized view	Yes	Pipeline update, <code>REFRESH</code> or schedule; incremental when possible	Gold aggregates for BI
Streaming table	Yes	Processing each new input row once	Ingestion, append-style layers

```
CREATE MATERIALIZED VIEW main.gold.daily_revenue AS
SELECT order_date, SUM(total_amount) AS revenue FROM main.silver.orders GROUP BY order_date;
```

3.7 Data quality checks

Where in the course: `07_lakeflow_pipelines` (expectations, quarantine), `lab_07_lakeflow_pipeline` Tasks 2, 6, 7, `03_delta_lake` (constraints), `06_medallion_architecture`

- Without an `ON VIOLATION` clause, the row is kept and the violation is recorded as a metric. In Python, the decorators are `@dp.expect`, `@dp.expect_or_drop`, `@dp.expect_or_fail` and the `expect_all*` variants.

- Metrics appear in the pipeline UI and in `event_log(TABLE(...))` under `details:flow_progress.data_quality.expectations`. Dropped rows are not stored, so write a quarantine table with the inverted rule if you need them.
- Delta constraints reject the whole write: `ALTER TABLE t ADD CONSTRAINT c CHECK (amount > 0)` or `SET NOT NULL`.
- Also check key uniqueness, referential integrity with a `left_anti` join, row counts between layers, and freshness.

```
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id      EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT positive_amt  EXPECT (total_amount > 0),           -- warn only
  CONSTRAINT known_status  EXPECT (status IN ('NEW','PAID')) ON VIOLATION FAIL UPDATE
) AS SELECT * FROM STREAM(bronze_orders);
```

SECTION 4

Working with Lakeflow Jobs (16%)

4.1 Control flow: retries, branching, looping

Where in the course: `08_job_orchestration` (options and retries; control flow as code: retries, If/else, For each), `lab_08_orchestration` Tasks A, B, D

- **Retries** are set per task (count, minimum interval, retry on timeout). Use them for transient errors, not for bugs or bad data.
- **If/else condition** compares operands with `==`, `!=`, `>`, `>=`, `<`, `<=`. Operands can be job parameters or task values such as `{{tasks.validate.values.row_count}}`. Downstream tasks depend on the true or the false branch.
- Task values: `dbutils.jobs.taskValues.set(key="row_count", value=n)` and `dbutils.jobs.taskValues.get(taskKey="validate", key="row_count", default=0, debugValue=0)`. `dbutils.notebook.exit()` does not create task values.
- **For each** runs a nested task for every element of a JSON array (read as `{{input}}`), with configurable concurrency.
- **Run if:** *All succeeded* (default), *At least one succeeded*, *None failed*, *All done* (cleanup), *At least one failed* (alerts), *All failed*. A **repair run** reruns only failed and skipped tasks and their dependants.

4.2 Task types and dependencies

Where in the course: `08_job_orchestration` (task types, multi-task job, medallion job YAML), `lab_08_orchestration` Section 1, `materials/cicd/resources/retailhub_job.yml`

- In scope: **Notebook**, **SQL** (a query, alert or file run on a SQL warehouse), **Dashboard** (refreshes an AI/BI dashboard and notifies subscribers) and **Pipeline** (runs a pipeline update). Others include Python script, wheel, JAR, Run Job and dbt.
- The DAG is defined with `depends_on`; tasks without dependencies on each other run in parallel.
- Job parameters reach every task (`dbutils.widgets.get, {{job.parameters.name}}`); dynamic values include `{{job.run_id}}` and `{{job.start_time.iso_date}}`.

- **Compute:** serverless by default for notebooks; job clusters can be shared by tasks; SQL and dashboard tasks need a SQL warehouse. By default one run is active at a time, and runs above the limit are skipped.

4.3 Schedules and trigger types

Where in the course: `08_job_orchestration` (triggers and schedule, cron expressions), `lab_08_orchestration` Step 7, Task C, closing check, `materials/orchestration/*_trigger_demo.py`

- Quartz cron starts with seconds: `0 0 2 * * ?` runs daily at 02:00, and `0 0 8 ? * MON-FRI` runs on weekdays at 08:00.
- File arrival needs `READ FILES` on the location. Table update triggers pass `{{job.trigger.table_update.updated_tables}}` to tasks. Bundles in mode: `development` pause triggers.

Trigger	Starts a run when	Settings
Scheduled	A time is reached	Quartz cron, time zone, pause
File arrival	New files land in a volume or external location	Minimum time between triggers, wait after last change
Table update	Unity Catalog tables receive commits	Any or all tables, minimum interval, wait after change
Continuous	The previous run ends	Backoff on failure

4.4 Time-based or data-driven triggers

Where in the course: `08_job_orchestration` (triggers), `lab_08_orchestration` Step 7 and Task C

- **Scheduled** when there is a fixed deadline or SLA, or the source loads at a known time; cost and timing are predictable.
- **File arrival** when files arrive irregularly and must be processed soon, without empty runs.
- **Table update** when a job depends on a table maintained by another pipeline or team, which avoids starting before the data is ready.
- **Continuous** job or pipeline for always-on, near-real-time processing. Dependent steps belong in one job as tasks, not as separate triggers.
- Batch frequent arrivals with *minimum time between triggers* and *wait after last change*; do not poll in a notebook loop.

SECTION 5

Implementing CI/CD (10%)

5.1 Git folders: branches, commit, push, pull requests

Where in the course: `09_cicd_and_automation` (Git folders UI walkthrough), `lab_09_dabs` (optional Git folder workflow), `01_platform_and_workspace`

- Create a Git folder in **Workspace** under **Create > Git folder** with the repository URL. Git credentials live in user settings under linked accounts.

- The **Git dialog** (branch button next to a notebook or folder) lets you switch branches, **Create Branch** from a base branch, **Pull**, review diffs, write a commit message and **Commit & Push**. Merge, rebase and reset are in its menu.
- Pull requests are opened, reviewed and merged in the Git provider (GitHub, Azure DevOps, GitLab) after you push a feature branch. CI then runs `databricks bundle validate` and `deploy`.
- Each developer should have their own Git folder and branch: switching the branch of a shared folder switches it for everyone.

5.2 Bundle variables and target overrides

Where in the course: 09_cicd_and_automation (target overrides), lab_09_dabs Tasks 1, 2, 6, materials/cicd/databricks.yml

- Reference variables as `${var.catalog}`, alongside substitutions like `${bundle.target}` and `${resources.pipelines.<key>.id}`. A lookup resolves IDs by name, for example of a warehouse.
- Precedence, highest first: `--var="catalog=x"`, `BUNDLE_VAR_catalog`, `.databricks/bundle/<target>/variable-overrides.json`, target variables, then default.
- A target overrides top-level mappings (workspace, resources, run_as, presets). `mode: development` prefixes names with `[dev <user>]` and pauses triggers; `mode: production` enforces safe settings.
- Variables resolve at deployment; values that change per run belong in job parameters.

```
variables:
  catalog:
    default: retailhub_dev
targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
    variables:
      catalog: retailhub_prod
    run_as:
      service_principal_name: "000000000-0000-0000-0000-000000000000"
```

5.3 Deploying bundles

Where in the course: 09_cicd_and_automation (bundle structure, deploy checklist), lab_09_dabs Tasks 3–5, materials/cicd/

- A bundle is `databricks.yml` plus included resource files defining jobs, pipelines, dashboards, schemas, volumes and more, together with their source code.
- `bundle deploy` uploads files to the target root path, creates or updates resources and tracks state. Resources removed from the configuration are deleted on the next deploy.
- Promote the same commit to `-t dev`, `-t test` and `-t prod`; only variables and overrides differ. References such as `${resources.pipelines.retailhub_pipeline.id}` wire tasks to pipelines in every target.
- `bundle init` starts from a template; `bundle generate job --existing-job-id <id>` and `bundle deployment bind adopt` existing resources.

5.4 Databricks CLI in CI/CD

Where in the course: 09_cicd_and_automation (CLI setup and authentication, CI integration), lab_09_dabs Task 3, materials/cicd/README.md

- The current CLI is a single binary (0.2xx or later); the legacy databricks-cli pip package has no bundle support.
- CI authenticates as a service principal with OAuth M2M (DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET) or workload identity federation. The GitHub action databricks/setup-cli installs the CLI.
- Other command groups manage assets directly: databricks jobs, pipelines, fs, repos.

Command	Purpose
databricks auth login --host <url> --profile TRAINING	User OAuth login stored as a profile
databricks bundle validate -t prod	Check configuration and variables; first step in CI
databricks bundle deploy -t prod	Upload and create or update resources
databricks bundle run -t prod retailhub_job	Run a bundle job or pipeline
databricks bundle summary / destroy	Show deployed resources / delete them

SECTION 6

Troubleshooting, Monitoring, and Optimization (10%)

6.1 Run history and baselines

Where in the course: BONUS_troubleshooting (run-history trends, system tables), 08_job_orchestration (monitoring via system tables), lab_08_orchestration Task E

- **Jobs & Pipelines**, then the job and its **Runs** tab: the matrix view shows total duration and status per run and the duration of every task; the list view shows start time, duration, trigger and result.
- Compare a run with a baseline, such as the median of recent successful runs, not just with the previous run. Gradual growth suggests data growth or missing layout maintenance. A step change after a deployment points to the change. Isolated outliers are often startup, queueing or retries.
- Duration warning thresholds with notifications flag slow runs. For long-term trends, query `system.lakeflow.job_run_timeline`, grouping by `run_id` because long runs span several rows.

```
SELECT job_id, run_id, MIN(period_start_time) AS started,
       timestampdiff(SECOND, MIN(period_start_time), MAX(period_end_time)) AS duration_s,
       MAX(result_state) AS result_state
FROM system.lakeflow.job_run_timeline
WHERE period_start_time > current_timestamp() - INTERVAL 30 DAYS
GROUP BY job_id, run_id ORDER BY job_id, started DESC;
```

6.2 Pipeline health in the Jobs UI

Where in the course: lab_troubleshooting Tasks 1 and 5, lab_08_orchestration Tasks D and E, lab_07_lake-flow_pipeline (monitoring)

- Run states: Queued, Running, Succeeded, Failed, Timed out, Canceled, Skipped (concurrency limit reached).
- In the task graph, the first failed task is the root cause. *Upstream failed* tasks never started; *Excluded* tasks had an unmet Run if condition. Read the first lines of the failed task's error.
- The jobs list shows recent run outcomes; system tables give failure rates over time. For pipelines, check the graph, the event log and data quality metrics.

6.3 Spark UI: skew, shuffle, spill

Where in the course: BONUS_troubleshooting (reading the Spark UI), 04_delta_optimization (performance bottlenecks), lab_troubleshooting Tasks 3 and 4

Open **Spark UI**, then **Stages**, then *Summary Metrics* (min, 25th percentile, median, 75th percentile, max). On serverless compute, use the query profile.

AQE splits a partition that is at least five times the median and larger than 256 MB. Adding executors does not split one oversized partition, and fewer shuffle partitions make skew worse.

Problem	Signature	Fix
Skew	Max task time and shuffle read far above the median	AQE skew join (on by default); broadcast; salt the hot key
Heavy shuffle	Large shuffle read and write at Exchange	Broadcast small tables; filter and prune columns early
Spill	<i>Spill (Memory)</i> and <i>Spill (Disk)</i> above zero	More partitions or auto; memory-optimised nodes; remove skew
Tiny tasks	Thousands of millisecond tasks	Fewer partitions or auto; AQE coalescing

6.4 Liquid clustering and predictive optimization

Where in the course: 04_delta_optimization (liquid clustering, predictive optimization), lab_04_optimization Tasks 3 and 4

- Liquid clustering replaces partitioning and Z-ordering (they cannot be combined), allows up to four keys, and lets you change keys without rewriting data. Choose filter and join columns.
- `CLUSTER BY AUTO` picks keys from query history on Unity Catalog managed tables with predictive optimization.
- Predictive optimization runs `OPTIMIZE`, `VACUUM` and `ANALYZE` on managed tables, can be set at catalog or schema level, is on by default for new accounts, and logs to `system.storage.predictive_optimization_operations_history`.

```
CREATE TABLE main.silver.orders CLUSTER BY (customer_id, order_date) AS SELECT * FROM
main.bronze.orders;
ALTER TABLE main.silver.orders CLUSTER BY AUTO;
OPTIMIZE main.silver.orders;                -- FULL reclusters all records
ALTER CATALOG main ENABLE PREDICTIVE OPTIMIZATION;
```

6.5 Startup failures, library conflicts, out of memory

Where in the course: `BONUS_troubleshooting` (failure patterns), `lab_troubleshooting` Task 6, `01_platform_and_workspace` (driver and executors)

Triage: no task time means infrastructure, so check the event log. An error before any stage means code, permissions or paths, so read the first error line. Slow or failing stages mean data or performance, so check stage metrics.

Problem	Signs and where to look	Fix
Compute does not start	Failure with no task time; init script or cloud launch error; compute Event log	Fix the init script; change VM size or region; raise quota; instance pools; serverless
Library conflict	<code>ModuleNotFoundError</code> or <code>AttributeError</code> after a library change; pip errors in the event log	Pin versions; notebook-scoped <code>%pip install pkg==x</code> plus <code>dbutils.library.restartPython()</code> ; serverless environments
Driver OOM	"Spark driver has stopped unexpectedly" after <code>collect()</code> or <code>toPandas()</code>	Write to tables; <code>limit()</code> ; avoid large broadcasts; bigger driver last
Executor OOM	<code>OutOfMemoryError</code> , lost executors, spill; skew or row explosion in the plan	Fix skew and join keys; more partitions; lower the broadcast threshold

SECTION 7

Governance and Security (15%)

7.1 Managed and external tables

Where in the course: `10_governance_and_security` (managed vs external lifecycle, converting between them), `03_delta_lake`, `BONUS_external_connection`, `guide_external_connection`

- Managed tables store data in Unity Catalog managed storage and get predictive optimization. `DROP TABLE` can be undone with `UNDROP` for 7 days; then the files are deleted.
- External tables use a path in an external location (`CREATE EXTERNAL TABLE` privilege). `DROP TABLE` removes metadata only and the files remain.
- `SET MANAGED` requires table ownership and Runtime 17.3 LTS or serverless compute; it copies data into managed storage, keeps name, history and grants, and requires streams to be restarted.
- There is no managed-to-external command: create an external table with `CREATE TABLE ... LOCATION '...' AS SELECT`, repoint consumers, then drop the managed table. Modify either type with `ALTER TABLE ... RENAME TO`, `SET TBLPROPERTIES` and `SET OWNER TO`.

```
CREATE TABLE main.silver.customers (id BIGINT, email STRING);           -- managed
CREATE TABLE main.bronze.orders_ext (id BIGINT, amount DECIMAL(10,2))  -- external
  LOCATION 'abfss://external@stdeatraining.dfs.core.windows.net/orders';
ALTER TABLE main.bronze.orders_ext SET MANAGED;                       -- external to managed
ALTER TABLE main.bronze.orders_ext UNSET MANAGED;                     -- rollback within 14 days
```

7.2 GRANT, REVOKE and DENY

Where in the course: 10_governance_and_security (access management, REVOKE and DENY), lab_10_governance Tasks 1, 7, 9

- Access is denied by default. Reading a table needs `SELECT` on it (or a parent) plus `USE CATALOG` and `USE SCHEMA`. Grants on a catalog or schema cover current and future children; metastore grants do not inherit.
- Principals: users (email), account groups and service principals (application ID), in backticks. Workspace-local groups cannot receive Unity Catalog grants.
- Owners, metastore admins and holders of **MANAGE** can grant. In the UI, go to **Catalog Explorer**, open the object, then **Permissions**. **REVOKE** removes only that grant; access may remain through a group or a parent object.
- **DENY** works only for legacy `hive_metastore` table ACLs, where it overrides **GRANT**. Unity Catalog has no **DENY** on data privileges: revoke where the privilege was granted, grant more narrowly, or use filters, masks and ABAC. ABAC DENY policies (Beta) can deny only **MANAGE ACCESS CONTROL**.

```
GRANT USE CATALOG ON CATALOG retail TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA retail.gold TO `analysts`;
GRANT MODIFY ON TABLE retail.silver.orders TO `6f1b2c3d-1111-2222-3333-444455556666`; -- service principal
REVOKE SELECT ON SCHEMA retail.gold FROM `analysts`;
SHOW GRANTS ON SCHEMA retail.gold;
```

7.3 Column masks and row filters

Where in the course: 10_governance_and_security (native masks and row filters, dynamic views), lab_10_governance Tasks 3–6, 8

- A filter function returns `BOOLEAN`; a mask returns the column's type. They apply to every query, including those through views and those run by the owner. Remove them with `DROP MASK` and `DROP ROW FILTER`.
- A table has one row filter at most, and a column one mask at most. They are listed in `information_schema.row_filters` and `information_schema.column_masks`.
- Dynamic views (`CASE WHEN is_account_group_member(...)`) are the older approach and protect only queries through the view.

```
CREATE FUNCTION retail.sec.mask_email(email STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN email ELSE '***' END;
ALTER TABLE retail.silver.customers ALTER COLUMN email SET MASK retail.sec.mask_email;

CREATE FUNCTION retail.sec.region_filter(region STRING)
RETURN is_account_group_member('admins') OR region = 'EMEA';
ALTER TABLE retail.silver.orders SET ROW FILTER retail.sec.region_filter ON (region);
```

7.4 ABAC policies

Where in the course: 10_governance_and_security (attribute-based access control)

- ABAC has been generally available since April 2026. **Governed tags** are defined at account level with allowed values and an **ASSIGN** permission. **Policies** on a catalog, schema or table bind a mask or row filter function to every matching column or table, including tables created later.

- Clauses: **TO** and **EXCEPT** name principals; **WHEN** `has_tag_value(...)` selects tables; **MATCH COLUMNS ... AS** `alias` selects columns; **ON COLUMN** names the masked column; **USING COLUMNS (...)** passes row filter arguments. The older `hasTagValue` spelling still works.
- One policy replaces per-table masks. Stewards tag data; security administrators write policies.
- Requirements: serverless compute or Runtime 16.4 and later; **MANAGE** on the securable and **EXECUTE** on the function. Inspect policies with **SHOW POLICIES** and **DESCRIBE POLICY**.

```
ALTER TABLE retail.silver.customers ALTER COLUMN ssn SET TAGS ('pii' = 'ssn');
CREATE POLICY redact_ssn ON SCHEMA retail.silver
COLUMN MASK retail.sec.redact
TO `account users` EXCEPT `compliance`
FOR TABLES
MATCH COLUMNS has_tag_value('pii', 'ssn') AS ssn_col
ON COLUMN ssn_col;
```

PRACTICE

Sample question patterns

Exam questions are short scenarios with four options. These patterns paraphrase typical question styles; they are not official questions.

Signal in the question	Likely answer	Why the alternatives fail
One task ten times slower; max shuffle read far above median	AQE skew join handling, broadcast or salting	More executors do not split a partition
Rollback after bad ingest, audit trail, one copy for BI and AI	Delta Lake time travel governed by Unity Catalog	File copies are not transactional or governed
Many concurrent analysts, fast start, controlled cost	Serverless SQL warehouse	Job and single-node clusters do not fit
Files keep arriving; schema changes	Auto Loader with schema location and evolution mode	Plain reads reload everything
Salesforce or SQL Server hourly, no engineering budget	Lakeflow Connect managed connector	JDBC means building CDC and retries yourself
Reruns must not duplicate data, SQL team	COPY INTO	CTAS reloads all files
Start only when an upstream table changes	Table update trigger	A schedule may run before data is ready
Rerun only the failed part of a job	Repair run	A full rerun repeats successful work
Same code in dev, test and prod with different catalogs	Bundle variables and targets, CLI in CI	Manual copies are not repeatable
Zero task time, init script error	Compute event log	No code ran, so the Spark UI is empty
Hide a column except for one group on every query	Column mask with <code>is_account_group_member</code>	A view protects only its own queries
One masking rule for hundreds of PII tables	ABAC policy on governed tags	Per-table masks miss new tables
Remove read access in Unity Catalog	REVOKE where it was granted	DENY is not supported in Unity Catalog
Dashboard aggregate refreshed on a schedule	Materialized view	Views recompute per query

When two answers seem right, prefer the managed, declarative and governed option unless the question states a requirement it cannot meet.

PLAN

Two-week study plan

About 1.5 hours on weekdays and 3 hours on weekend days. If practice scores stay below 80% in week 2, move the exam date.

Day	Focus	Activities
1	Sections 1, 2.1-2.3	Reread <code>01_platform_and_workspace</code> , <code>05_incremental_ingestion</code> ; redo <code>lab_05_streaming</code>
2	Sections 2.4-2.7	<code>02a_lakeflow_connect_demo</code> , the Lakeflow Connect guide; write a JDBC and a REST landing notebook
3	Sections 3.1-3.4	Redo <code>lab_06_transformations</code> without the guide; joins, unions, dedup in PySpark and SQL
4	Sections 3.5-3.7	One tuning experiment with the query profile; <code>lab_07_lakeflow_pipeline</code> Tasks 2-7
5	Section 4	Rebuild the <code>lab_08_orchestration</code> job: retries, If/else, For each, triggers, repair
6	Sections 5-6	Deploy <code>materials/cicd</code> with the CLI; <code>lab_troubleshooting</code> , <code>BONUS_troubleshooting</code>
7	Section 7	Redo <code>lab_10_governance</code> ; write an ABAC policy from memory; course quizzes
8	Weak areas	Map every wrong quiz answer to an objective and reread it here
9	Official courses	Databricks Academy self-paced courses from the exam guide for weak domains
10	Practice exam 1	Timed, 90 minutes; review every answer
11	Revision	Sections below 80%; sample question patterns
12	Hands-on	One lab per weak domain, in a workspace or Databricks Free Edition
13	Practice exam 2	Confirm at least 80%; check the current exam guide
14	Light review	The bullet points in this map only; check booking and ID requirements